



ECOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE ET D'ANALYSE DES
SYSTÈMES - RABAT

DEPARTEMENT WEB AND MOBILE ENGINEERING

Field : IDSIT

Second Year Internship

Design and Implementation of a High-Availability Kubernetes Platform for Production-Grade Application Deployment

Realised by :

TAMASNA ANOUAR

Jury :

Pr. Khalid NAFIL

Pr. Karima MOUMANE

Academic year 2024-2025

Dedication

I dedicate this work to:

My mother, sources of tenderness and love, for her support throughout my school life.

My father, who has always supported me and did everything possible to help me.
my brothers and sisters, whom I love very much.

My extended family.

My dear friends and teachers.

All those who, from near or far, contributed to the development of this work.

May God grant them health and prosperity.

Acknowledgements

First of all, praise to God, Who guided me on the right path throughout the work and inspired me with the right steps and the right reflexes. Without His mercy, this work would not have come to fruition.

I would like to extend our most sincere thanks to the members of the jury. Please accept in this work my sincere respect and deep gratitude.

I would also like to thank Miss Nessrine LAFHAL very much, who, as supervisors of the project, were always attentive despite their impediments. their help was precious, as was the time they devoted to me.

I do not forget my parents for their contributions, their support, and their patience.

Finally, I would like to thank all my relatives and friends, who have always supported and encouraged me during the realization of this project.

Thank you, everyone.

Summary

This report reflects the culmination of my work during my internship at Digital Data Service, where I successfully designed and deployed a high-availability Kubernetes infrastructure. The primary objective was to create a resilient and scalable environment capable of hosting production-grade applications while ensuring reliability, fault tolerance, and streamlined DevOps workflows.

The infrastructure leverages a modern cloud-native stack, including Rancher for cluster management, ArgoCD for GitOps-driven continuous delivery, and Cert-Manager for automated TLS certificate management. Core services such as Harbor (private container registry with security scanning), MinIO (object storage), Redis (high-availability caching), PostgreSQL HA (with pgAdmin), and Apache Tika were deployed and configured with persistent storage to guarantee data durability.

Ingress was managed through NGINX controller, enabling secure external access via custom domains. The solution was designed with automation, scalability, and maintainability in mind, supporting dynamic workloads and reducing operational overhead.

Throughout the project, I focused on ensuring high availability, performance optimization, and security, while integrating DevOps best practices to meet the requirements of both developers and system administrators.

Key-words: Kubernetes HA, GitOps / ArgoCD, Rancher, CI/CD, DevOps

Résumé

Ce rapport reflète l'aboutissement de mon travail réalisé durant mon stage au sein de Digital Data Service, où j'ai conçu et déployé une infrastructure Kubernetes haute disponibilité. L'objectif principal était de mettre en place un environnement résilient et évolutif, capable d'héberger des applications de niveau production tout en garantissant fiabilité, tolérance aux pannes et automatisation des workflows DevOps.

L'infrastructure s'appuie sur un écosystème moderne cloud-native, incluant Rancher pour la gestion des clusters, ArgoCD pour le déploiement continu basé sur GitOps, et Cert-Manager pour la gestion automatisée des certificats TLS. Des services clés tels que Harbor (registre privé de conteneurs avec analyse de sécurité), MinIO (stockage objet), Redis (cache haute disponibilité), PostgreSQL HA (avec pgAdmin), et Apache Tika ont été déployés et configurés avec un stockage persistant afin d'assurer la durabilité des données.

L'accès externe a été géré via le contrôleur NGINX, permettant une exposition sécurisée des applications à travers des domaines personnalisés. La solution a été conçue dans une optique d'automatisation, de scalabilité et de maintenabilité, tout en réduisant la charge opérationnelle.

Tout au long du projet, je me suis concentré sur la garantie de la haute disponibilité, l'optimisation des performances et la sécurité, en intégrant les bonnes pratiques DevOps afin de répondre aux besoins des développeurs.

Abbreviations List

Abbreviation	Designation
HA	High Availability
CI/CD	Continuous Integration / Continuous Deployment
PVC	Persistent Volume Claim
DNS	Domain Name System
GitOps	Git-based Operations
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secured

Table Of Contents

Dedication	I
Thanks	II
Summary	III
Résumé	IV
Abbreviations List	V
General Introduction	1
Chapter I : Work Context	3
1.1 Internship Environment	4
1.2 Project Presentation	5
1.2.1 Problem Statement	5
1.2.2 Proposed Solutions	6
1.3 Project Planning (Gantt Chart):	7
1.3.1 Gantt Chart	7
1.4 Conclusion	8
Chapter II: Needs Analysis	9
2.1 Introduction	10
2.2 Needs Identification	10
2.3 Functional Requirements Specification:	11
2.4 Non-Functional Requirements Specification:	12
2.5 Conclusion	12
Chapter III: Analysis and Design	14
3.1 Infrastructure Component Diagram	15
3.2 Application Deployment Workflows	17
3.2.1 HealthCore Workflow	17
3.2.2 NeuroTalk Workflow	18
3.3 Conclusion	19
Chapter IV: Implementation	20
4.1 Tools Used	21
4.1.1 Software Resources:	21
4.1.2 Programming Languages:	23
4.1.3 Supporting Tools:	24
4.2 Set up and Configuration	25
4.2.1 Cluster Dashboard (Rancher)	25
4.2.2 Projects and Namespaces	25

4.2.3	Container Registry (Harbor)	26
4.2.4	GitOps Dashboard (Argo CD)	27
4.2.5	Object Storage (MinIO)	28
4.2.6	Relational Database (PostgreSQL)	29
4.2.7	Persistent Volumes (PVCs)	29
4.2.8	Workload Layout (NeuroTalk stack)	30
4.2.9	MinIO Kubernetes Manifests	30
4.2.10	PostgreSQL and pgAdmin Manifests	30
4.2.11	Security and Operations Highlights	31
4.3	Benchmark	31
4.4	Conclusion	33
	General Conclusion	33

List of Figures

1.1	Digital Data Service — Company Profile	5
1.2	GANTT chart	8
3.3	Infrastructure Component Diagram	16
3.4	HealthCore Workflow Diagram	18
3.5	NeuroTalk Workflow Diagram	19
4.6	Rancher cluster dashboard overview	25
4.7	Logical isolation via Projects/Namespaces	26
4.8	Harbor projects used for image management	27
4.9	Argocd Overview	28
4.10	MinIO Object Store Console	28
4.11	pgAdmin attached to the PostgreSQL cluster	29
4.12	Cluster-wide PVC status for stateful workloads	29
4.13	NeuroTalk stack directory structure	30
4.14	MinIO Kubernetes manifests	30
4.15	PostgreSQL and pgAdmin deployment resources	31
4.16	Cluster dashboard snapshot (Rancher)	32

General Introduction

In recent years, the adoption of **cloud-native technologies** and **Kubernetes orchestration** has transformed the way organizations deploy and manage applications. Ensuring **high availability, scalability, and fault tolerance** has become essential for production-grade workloads, particularly for businesses that rely on continuous service delivery and resilient infrastructure.

Our internship took place at **Digital Data Service**, where we worked on designing and deploying a **high-availability Kubernetes infrastructure**.

This environment supports the deployment of multiple production-grade services, including **ArgoCD** for GitOps-driven continuous delivery, **Rancher** for cluster management, **Harbor** as a private container registry, **MinIO** for object storage, **Redis** for caching, **PostgreSQL HA** for relational data management, **Apache Tika** for document processing, and two key company services, **NeuroTalk** and **HealthCore**.

We also implemented **NGINX Ingress controllers** for secure external access and configured **persistent storage** for all critical services to ensure **data durability**.

The presentation of this report is structured into **four chapters**.

The first chapter presents the internship environment, outlines the challenges addressed, introduces the proposed solution, and details the methodology followed.

The second chapter focuses on the **analysis of system components**, infrastructure requirements, and best practices for high availability and security, including the staging and deployment of **NeuroTalk** and **HealthCore**.

The third chapter covers the **design and architecture**, with detailed diagrams of the Kubernetes cluster, services, and networking setup.

Finally, the fourth chapter presents the **deployment and operational procedures**, including

configuration of persistent volumes, GitOps pipelines, monitoring tools, and deployment strategies for **NeuroTalk** and **HealthCore**.

It is important to note that this project involved **building a resilient and scalable platform from scratch**, automating deployments, ensuring **high availability and fault tolerance**, and integrating **security and monitoring solutions** to meet production standards while minimizing operational overhead, while also successfully staging and supporting **NeuroTalk** and **HealthCore**.

Chapter I

General Project Context

Introduction

In this first chapter, we delve into the context of our project, focused on the development and enhancement of a **high-availability Kubernetes infrastructure**.

Our initiative involves deploying a resilient platform with multiple production-grade services, including **ArgoCD**, **Rancher**, **Harbor**, **MinIO**, **Redis**, **PostgreSQL HA**, **Apache Tika**, and two key company services, **NeuroTalk** and **HealthCore**.

We also implemented secure **NGINX Ingress controllers** and configured persistent storage for all critical services to ensure **data durability**.

This work takes place in a dynamic professional environment at **Digital Data Service**, a company specializing in IT infrastructure and cloud-native solutions.

Our goals are to ensure **high availability, scalability, and fault tolerance** of deployed applications, streamline management of the infrastructure, implement robust monitoring and security practices, and provide reliable staging and deployment for key company services like **NeuroTalk** and **HealthCore**.

This introduction lays the essential foundations for understanding the scope and objectives of our project.

1.1 Internship Environment



Digital Data Service is a startup committed to supporting businesses in their digital journey. Through a diverse range of digital services tailored to the local context, we facilitate their technological transition in a strategic, agile, and customized manner.[1]

Company Name	Digital Data Service
Headquarters	Casablanca, Morocco
represented by	Mr. Anas TAHA
Staff	10 employees and 8 interns
Website	https://www.digitaldataservices.ma
Email	contact@digitaldataservices.ma
Phone	+212 777-077986

Figure 1.1: Digital Data Service — Company Profile

1.2 Project Presentation

1.2.1 Problem Statement

In modern IT environments, organizations face major challenges when deploying and managing complex infrastructures. Companies need to ensure **high availability, scalability, and fault tolerance** for their mission-critical applications, while also guaranteeing secure access, data durability, and efficient deployment pipelines.

Traditional infrastructures often suffer from issues such as downtime, limited scalability, inefficient communication between services, security vulnerabilities, and the absence of centralized monitoring and management solutions.

Moreover, deploying multiple production-grade services like **ArgoCD**, **Rancher**, **Harbor**, **MinIO**, **Redis**, and **PostgreSQL HA**, in addition to core company services such as **NeuroTalk** and **HealthCore**, introduces significant complexity that requires a robust, automated, and secure Kubernetes-based platform.

1.2.2 Proposed Solutions

Our project addresses these challenges by designing and deploying a fully integrated **high-availability Kubernetes infrastructure** that ensures resilience, scalability, and operational efficiency. The key solutions we implemented include:

- *Modern and user-friendly platform management:* We integrated **Rancher** for centralized cluster management, providing a clean and intuitive interface to handle multiple Kubernetes clusters securely and efficiently.

- *Continuous delivery and automation:* Using **ArgoCD**, we implemented GitOps practices, enabling automated, reliable, and version-controlled deployments for all applications and services.

- *Secure storage and artifact management:* We deployed **Harbor** as a private container registry and configured persistent volumes for all critical services such as **MinIO** and **PostgreSQL HA**, ensuring data durability and compliance with enterprise standards.

- *High availability and scalability:* The platform was built with multiple replicas, fault-tolerant deployments, and an **NGINX Ingress controller** for secure load balancing and traffic routing across services.

- *Monitoring, communication, and transparency:* We established a robust monitoring and logging stack to track service health and performance. Additionally, our approach allows clear visibility over deployed applications, ensuring transparency for both system administrators and developers.

- *End-to-end integration for company services:* NeuroTalk and HealthCore, as core business applications, were deployed on the platform with complete support for CI/CD pipelines, persistence, and scalability, enabling seamless operations and reliable staging environments.

Through these solutions, our project provides a secure, highly available, and scalable infrastructure that addresses the critical needs of modern enterprises while laying the foundation for future growth and innovation.

1.3 Project Planning (Gantt Chart):

1.3.1 Gantt Chart

Our project was managed by following a chronological breakdown into phases, which specify what must be done (tasks) and by whom (resources). To represent this distribution, we use a Gantt Chart, a type of bar chart that illustrates a series of tasks over a given period of time.

This chart provides a clear overview of the activities carried out within the project, including their start and end dates, and constitutes a visual representation of the project timeline.

In our case, the Gantt Chart highlights the different stages of deployment and testing in the Kubernetes environment (K3s cluster). It includes tasks such as setting up the cluster infrastructure, configuring networking and Ingress (without Traefik), ensuring persistence with MinIO, and staging two company services, namely **NeuroTalk** and **HeathCore**, before moving to production.

Here is an example of our **Gantt Chart** that illustrates the main milestones and phases of our project:

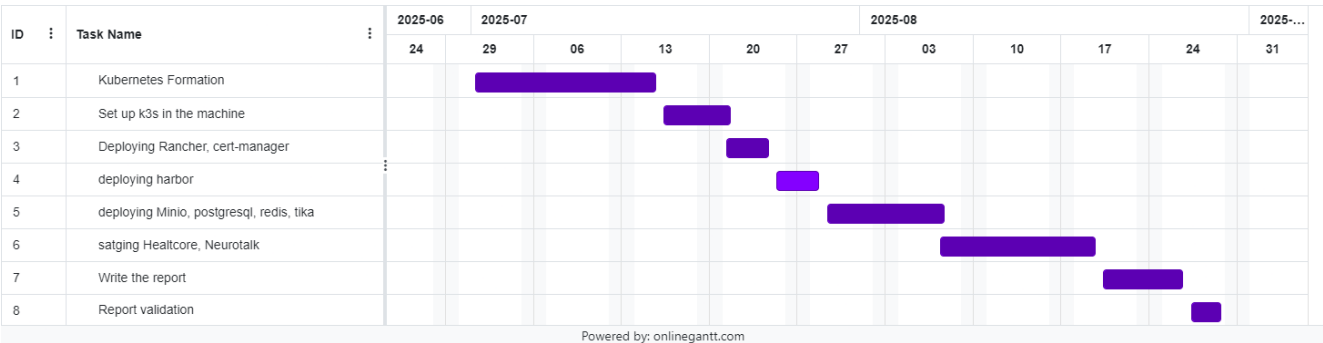


Figure 1.2: GANTT chart

1.4 Conclusion

This first chapter immersed us in the general context of our project, highlighting the working environment within **Digital Data Service (DDS)**. We presented the motivations behind our initiative, as well as the challenges related to infrastructure deployment, service management, and the need for a reliable and scalable environment.

We also introduced the core objectives of our work, including the deployment of a Kubernetes (K3s) cluster, the configuration of networking and persistence services, and the staging of two key company platforms: **NeuroTalk** and **HeathCore**. These steps constitute the foundation for a secure, efficient, and production-ready system.

In the next chapter, we will address a crucial phase: the **requirements analysis**. We will explore in depth the specific needs of users and stakeholders, both from a technical and business perspective. This stage is essential to design a solution that meets all expectations and requirements.

A detailed analysis of these needs will establish a solid foundation for the subsequent development and optimization of our project. Let us now move to the next chapter, where we will dive into this comprehensive study of requirements.

Chapter II

Needs Analysis

2.1 Introduction

The company operates in the field of digital data services and requires robust, reliable, and secure infrastructure to support its growing activities. The goal of this project is to design and deploy a solution that ensures high availability (HA), scalability, and optimized resource management for production workloads. This approach guarantees continuity of services, resilience against failures, and the ability to handle increasing demand efficiently.

2.2 Needs Identification

In the context of digital data services, the focus is not on end-users but rather on the infrastructure and operational requirements. The following needs have been identified as critical for the company's environment:

High Availability (HA): The system must be resilient and capable of running without service interruption, even in the case of node or component failure. This includes load balancing, failover mechanisms, and redundancy across services.

Scalability: The infrastructure should be able to scale both vertically (adding more resources to a node) and horizontally (adding more nodes to the cluster) to support increasing workloads as the company grows.

Monitoring and Observability: The company needs full visibility into system health, performance, and availability. This requires setting up centralized logging, metrics collection, and alerting systems to quickly detect and resolve incidents.

Automation and CD Pipelines: To streamline deployments and reduce manual intervention, infrastructure and application delivery must rely on automation tools and CD pipelines. This will ensure consistency, faster releases, and reduced risk of configuration errors.

Security and Compliance: Given the sensitive nature of digital data services, the deployment must include strong security practices such as secret management, encrypted communication, controlled access, and compliance with industry standards.

Cost Optimization: The infrastructure should balance performance and reliability with cost-effectiveness. This includes the use of containerization, resource quotas, and autoscaling policies to prevent over-provisioning.

Together, these requirements will guide the design and deployment of an infrastructure that meets the company's operational, business, and security objectives.

2.3 Functional Requirements Specification:

The infrastructure will be designed to provide the following functionalities:

High Availability Management: The deployment must guarantee continuous service availability through redundancy, clustering, and automated failover mechanisms. Even in the event of node or component failure, the system should remain fully operational.

Scalability and Elasticity: The solution should allow both vertical and horizontal scaling to handle increasing workloads. Autoscaling mechanisms will be implemented to allocate resources dynamically, ensuring efficiency without over-provisioning.

Monitoring and Observability: Centralized logging, monitoring, and alerting will be deployed to track system health, performance, and availability. This includes dashboards to visualize resource utilization, error rates, and real-time service status.

Automation and CD Pipelines: Deployment, updates, and configuration management must be automated through DevOps pipelines. This ensures consistency across environments, reduces human error, and accelerates release cycles.

Disaster Recovery and Backup: The infrastructure will include automated backups, recovery strategies, and geographic redundancy to minimize downtime and data loss in case of major incidents.

Security and Access Control: The solution must ensure secure communication, identity

management, and secret storage. Policies such as role-based access control (RBAC), TLS encryption, and compliance standards will be enforced.

Resource and Cost Management: Dashboards and tools will allow tracking of infrastructure usage and costs. Autoscaling and quota management will ensure optimized resource allocation.

2.4 Non-Functional Requirements Specification:

To ensure smooth operation and user satisfaction, several non-functional constraints must be taken into account during the design and implementation of the project:

- **Performance and Reliability:**

The system must be responsive and highly available, maintaining reliable performance under varying workloads.

- **Scalability:**

The solution should handle future growth seamlessly by scaling horizontally with additional nodes and services.

- **Security:**

Data integrity and confidentiality must be guaranteed. The system should include encryption, intrusion detection, secure APIs, and compliance with security standards.

- **Maintainability:**

The infrastructure code (Infrastructure-as-Code) must be well-structured, modular, and documented to facilitate future updates and enhancements.

- **Compatibility:**

The deployment must be compatible with multi-cloud and hybrid environments. It should support containerized workloads and be portable across different environments.

- **Monitoring and Transparency:**

The system must provide visibility into all layers of the infrastructure, with real-time metrics and logs accessible to the DevOps team.

- **Ease of Use:**

Interfaces such as dashboards and management consoles must be intuitive and easy for administrators to operate.

2.5 Conclusion

This second chapter allowed us to explore in detail the specific needs of the company, focusing on the deployment of a high availability and scalable infrastructure. We identified the main

challenges and proposed solutions addressing reliability, performance, and security requirements.

By concentrating on the needs analysis, this phase represents a crucial foundation for the design of a robust and resilient DevOps architecture. The detailed analysis conducted here provides a solid basis for the next phase of the project.

In the next chapter, we will move on to analysis and design, including architectural modeling and deployment planning. These steps will allow us to translate the identified requirements into concrete technical solutions.

Chapter III

Analysis and Design

Introduction

The creation of a robust and highly available platform must be preceded by a thorough analysis and design phase. This ensures that the development process aligns perfectly with the project's requirements.

In this chapter, we'll focus on the architectural design of our platform. We'll present general diagrams that model and describe the desired operation of our Kubernetes infrastructure. Two conceptual perspectives will be described: the first gives an overview of the platform's core components, and the second provides a detailed view of the application deployment workflows.

3.1 Infrastructure Component Diagram

The first step in our design process was to model the relationships between the core services of our Kubernetes cluster. This component diagram illustrates how **Harbor**, **Rancher**, **Ingress**, **Cert-Manager**, and other key elements interact to form a coherent and functional system.

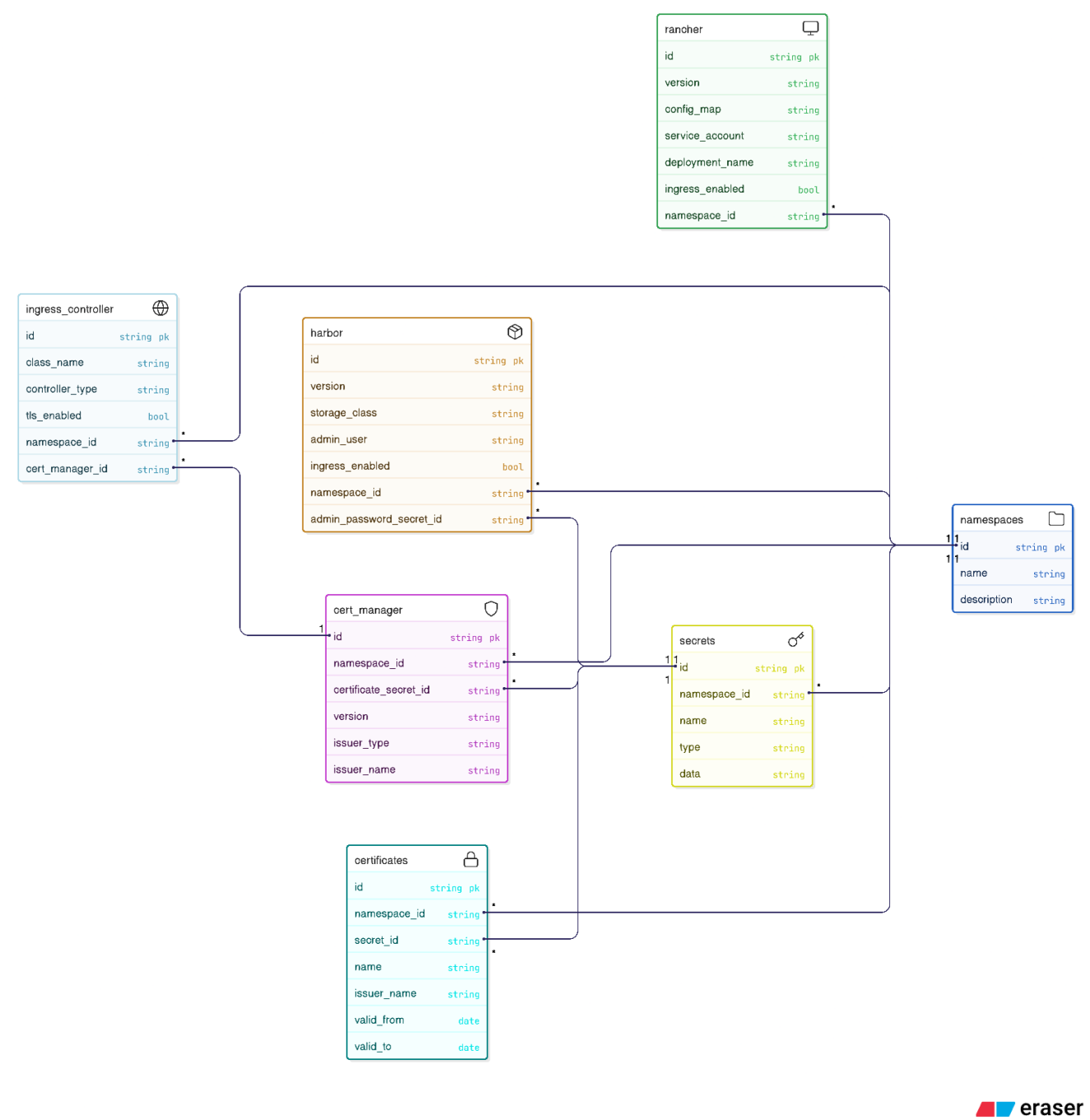


Figure 3.3: Infrastructure Component Diagram

As shown in the diagram, **Rancher** acts as the central management plane for our cluster, providing a user-friendly interface to manage multiple environments and namespaces. The **Ingress Controller** manages external access and routing, working in tandem with **Cert-Manager** to automatically provision and renew TLS certificates for secure communication. **Harbor** is our

private container registry, ensuring that all our application images are stored securely and are accessible by the cluster. These services all depend on **Secrets** and **Namespaces** to handle sensitive information and logically isolate resources within the cluster.

This high-level view helps us understand the logical dependencies and ensures a robust foundation for the entire infrastructure.

3.2 Application Deployment Workflows

To demonstrate the platform’s capabilities, we designed a **Continuous Integration/Continuous Deployment (CD)** pipeline for two of the company’s core applications: **HealthCore** and **NeuroTalk**. These workflow diagrams illustrate the end-to-end process, from code commit to application deployment, using a **GitOps** approach.

3.2.1 HealthCore Workflow

The HealthCore workflow diagram details how the application is built, deployed, and managed. The process begins in the **CD Pipeline**, where code changes in **GitLab** trigger a CI job. This job builds the application image and pushes it to our private registry, **Harbor**, and updates the deployment manifests in the Git repository.

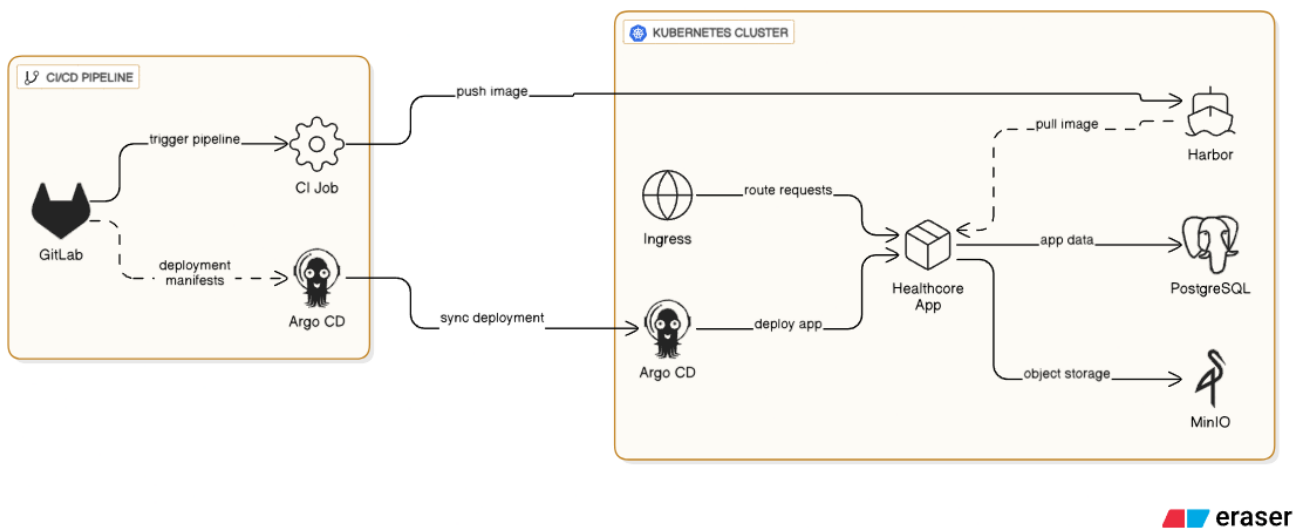


Figure 3.4: HealthCore Workflow Diagram

From there, **ArgoCD** takes over. As a GitOps tool, it constantly monitors the Git repository for changes. When a new commit with updated deployment manifests is found, ArgoCD automatically synchronizes the changes with the **Kubernetes Cluster**, pulling the new image from Harbor and deploying the **HealthCore App**. External traffic is then routed to the application through the **Ingress** controller. The application itself relies on persistent storage, connecting to a **PostgreSQL** database for application data and **MinIO** for object storage.

3.2.2 NeuroTalk Workflow

The NeuroTalk workflow is similar but includes more complex application services to handle specific tasks.

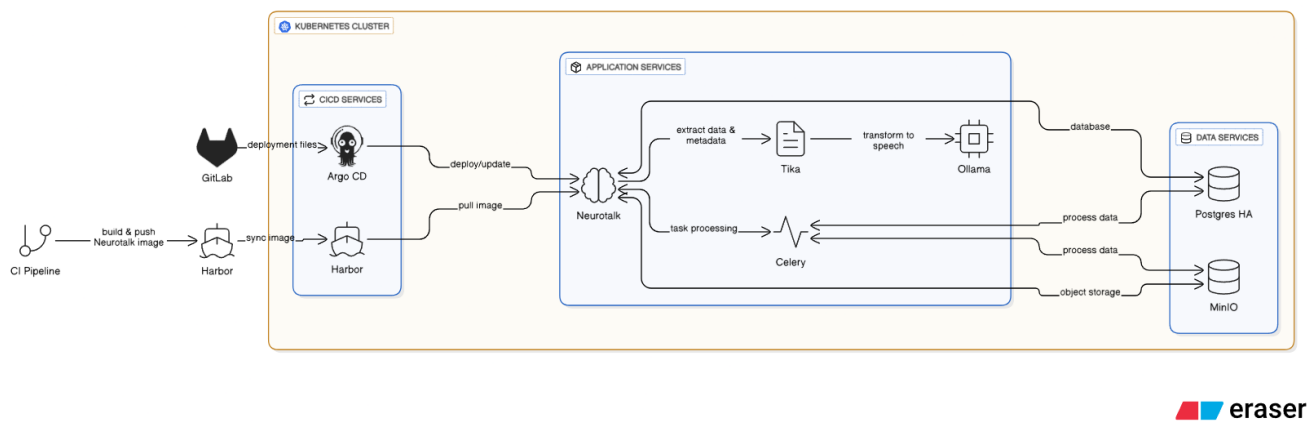


Figure 3.5: NeuroTalk Workflow Diagram

Just like with HealthCore, the **CI Pipeline** builds and pushes the application image to **Harbor**. **ArgoCD** then pulls the image and deploys the **NeuroTalk** application. Once deployed, the application interacts with several other services in the cluster: **Apache Tika** for document data extraction, **Ollama** for language processing, and **Celery** for asynchronous task processing. Data persistence is handled by a **PostgreSQL HA** (High-Availability) database and **MinIO** for object storage.

These diagrams effectively illustrate how our platform’s design supports a modern, automated, and scalable deployment model for production applications.

3.3 Conclusion

In this chapter, we transitioned from high-level requirements to a concrete architectural design. We used component diagrams to illustrate the logical relationships between our core services and workflow diagrams to detail the automated deployment process for our key applications. This design provides a solid foundation for a resilient and scalable infrastructure.

In the next chapter, we will move on to the practical implementation, detailing the operational procedures for deploying and configuring our Kubernetes cluster.

Chapter IV

Implementation

Introduction

After completing the analysis and design phase, this chapter focuses on the practical implementation of our work. We begin with a presentation of the main tools and technologies used (software resources, programming languages, frameworks, and supporting tools). Afterwards, we provide details about the features, configurations, and deployment of our high-availability Kubernetes infrastructure and the company services **NeuroTalk** and **HealthCore**.

4.1 Tools Used

4.1.1 Software Resources:

K3s: We chose **K3s**, a lightweight Kubernetes distribution, because it offers the same powerful orchestration capabilities as standard Kubernetes (scaling, self-healing, load balancing) with reduced complexity and resource requirements. It is well-suited for production deployments while remaining simple to operate.

How we use it: We deployed a multi-node K3s cluster that serves as the foundation of our platform, orchestrating **HealthCore** and **NeuroTalk** services with high availability and scalability.



[2]

Rancher: Rancher was selected because it provides a centralized management plane for Kubernetes clusters. It simplifies operations such as cluster provisioning, user management, and monitoring.

How we use it: We integrated Rancher to manage our K3s cluster with a user-friendly interface, control access rights, and provide administrators with full observability across namespaces and environments.



[3]

ArgoCD: We adopted ArgoCD for GitOps-based continuous delivery, as it ensures reliable and reproducible deployments. It was chosen for its ability to continuously synchronize the cluster state with Git repositories.

How we use it: ArgoCD automates deployments for **HealthCore** and **NeuroTalk** by pulling Kubernetes manifests and Helm charts from Git, ensuring that the live environment always matches the declared configuration.



[4]

Harbor: Harbor was chosen as a secure private container registry because it offers image scanning, access control, and integrity verification.

How we use it: All Docker images built from our CI pipelines are stored in Harbor before deployment. This guarantees trusted and verified images for our Kubernetes workloads.



[5]

NGINX Ingress Controller: We selected NGINX as the ingress controller due to its robustness, flexibility, and support for advanced routing and SSL termination.

How we use it: NGINX Ingress routes external traffic to the correct services inside the cluster. For example, requests to `neurotalk.digitalds.com` and `healthcore.digitalds.com` are securely routed through it, with certificates handled by Cert-Manager.



[6]

MinIO: MinIO was chosen for object storage because it provides S3-compatible APIs in a lightweight and cloud-native form.

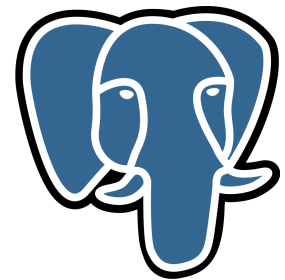
How we use it: Our applications rely on MinIO to store medical reports, documents, and unstructured datasets. It is deployed in distributed mode to ensure durability and scalability.



[7]

PostgreSQL HA: PostgreSQL was selected as the primary relational database because of its reliability, scalability, and advanced SQL support. We configured it in High-Availability mode to avoid downtime.

How we use it: PostgreSQL stores structured data for **HealthCore** and **NeuroTalk**. It runs as a StatefulSet with persistent volumes, and is paired with **PgBouncer** for efficient connection pooling.



[8]

4.1.2 Programming Languages:

YAML: Chosen for its readability and simplicity, YAML is the standard for defining Kubernetes resources.

How we use it: All deployments, services, ingress rules, and persistent volume claims are defined as YAML manifests and version-controlled in Git.



[9]

Bash: Bash was chosen for automation tasks because it is lightweight and ubiquitous across systems.

How we use it: Bash scripts automate cluster bootstrapping, application deployments, and CI/CD tasks for reproducibility.



[10]

4.1.3 Supporting Tools:

Helm: We selected Helm because it simplifies the deployment of complex applications with reusable charts.

How we use it: PostgreSQL HA, Harbor, and MinIO were deployed using official Helm charts, reducing manual configuration and ensuring consistent deployments.



[11]

4.2 Set up and Configuration

4.2.1 Cluster Dashboard (Rancher)

The platform runs on a K3s cluster managed via Rancher. The cluster dashboard provides an overview of resources (pods, CPU, memory) and the general health of the environment, which is essential for capacity planning and SRE operations.

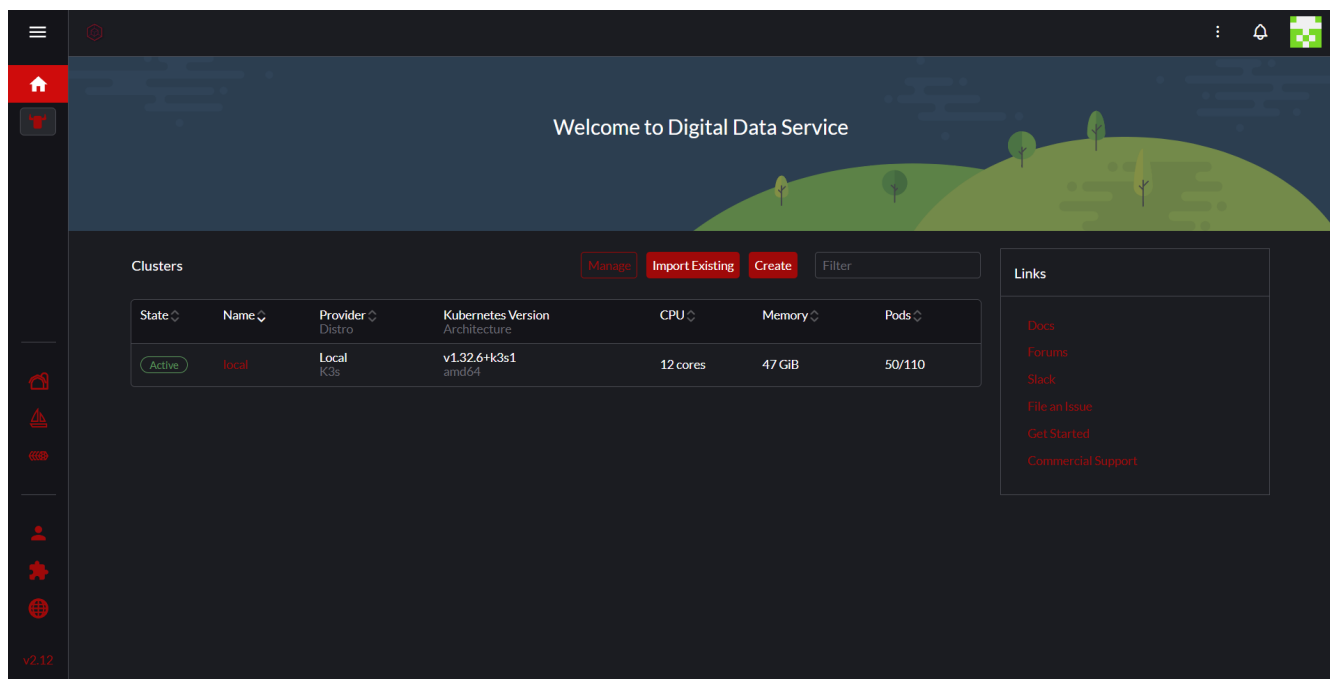


Figure 4.6: Rancher cluster dashboard overview

4.2.2 Projects and Namespaces

Rancher's Projects/Namespaces view is used to segment workloads (e.g., harbor, minio, postgres, redis, tika, neurotalk) and system components (e.g., cnpg-system). This enforces isolation, RBAC scoping, and resource quotas per team or service.

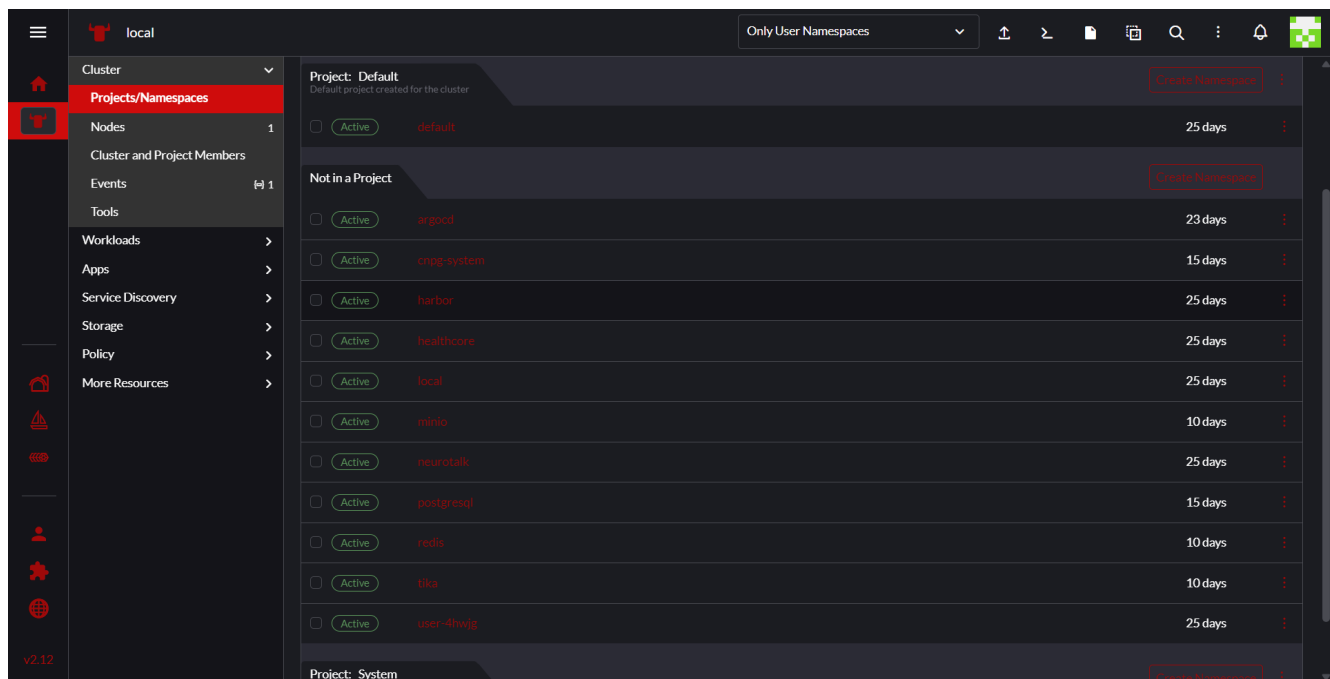


Figure 4.7: Logical isolation via Projects/Namespaces

4.2.3 Container Registry (Harbor)

Harbor hosts our private OCI images, organized by projects (e.g., healthcare, neurotalk-front, neurotalk-back, library). It provides vulnerability scanning, role-based access, and image retention policies to keep the registry clean and secure.

Project Name	Access Level	Role	Type	Repositories Count	Creation Time
healthcore	Private	Project Admin	Project	2	8/2/25, 12:40 PM
healthcore-backend	Private	-	Project	2	8/2/25, 7:55 PM
healthcore-front	Private	-	Project	1	8/2/25, 12:43 AM
library	Public	Project Admin	Project	0	7/28/25, 1:41 AM
neurotalk-back	Public	-	Project	1	8/12/25, 11:52 AM
neurotalk-front	Public	-	Project	1	8/12/25, 11:03 AM
neurotalk_front	Private	-	Project	0	8/12/25, 10:24 AM

Figure 4.8: Harbor projects used for image management

4.2.4 GitOps Dashboard (Argo CD)

Deployments are managed with GitOps using Argo CD. Each *Application* tracks a Git repository, path, and target revision, and continuously reconciles the live cluster state with the desired state from Git. Argo CD exposes two key signals: **Health** (e.g., Healthy/Degraded) and **Sync** (Synced/OutOfSync).

- Applications: `healthcore-backend` and `healthcore-frontend`
- Destination: `in-cluster` Namespace: `healthcore` Project: `default`
- Repos/Paths: Git (GitLab), paths `k8s/base/backend` and `k8s/base/frontend`
- Status (from snapshot): both **Healthy**; frontend **Synced**, backend **Synced**

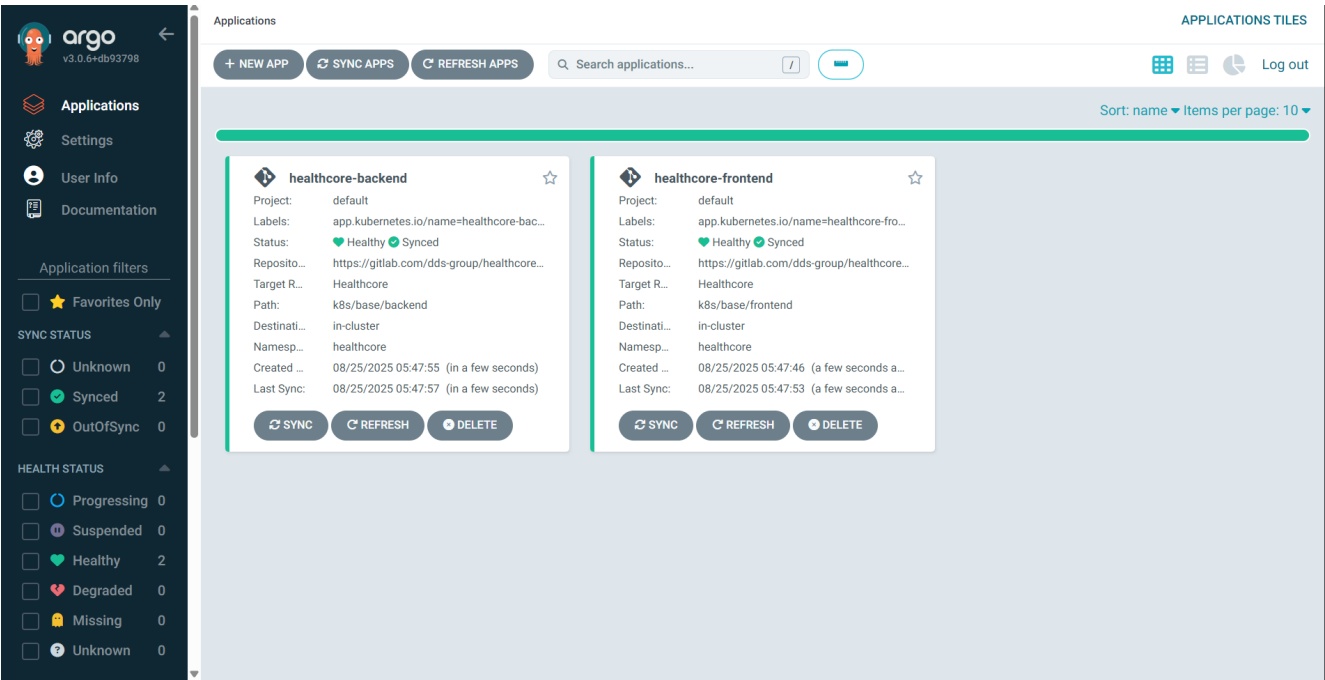


Figure 4.9: ArgoCd Overview

4.2.5 Object Storage (MinIO)

MinIO provides S3-compatible object storage used for application assets, backups, and model artifacts. Buckets and policies are managed through the MinIO Console.

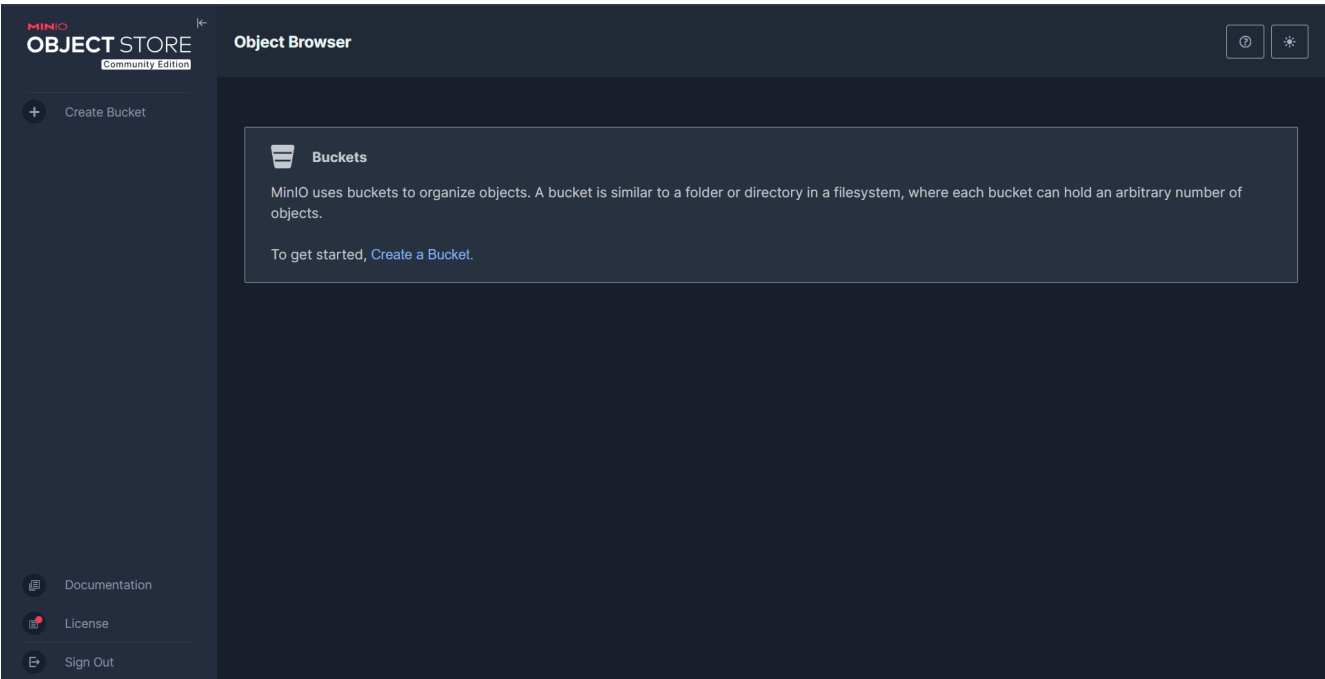


Figure 4.10: MinIO Object Store Console

4.2.6 Relational Database (PostgreSQL)

PostgreSQL backs transactional data. Administration is performed through pgAdmin for observability and maintenance (roles, schemas, backups). In production, we rely on Kubernetes-native HA patterns (e.g., CloudNativePG or replicated StatefulSets) with persistent volumes.

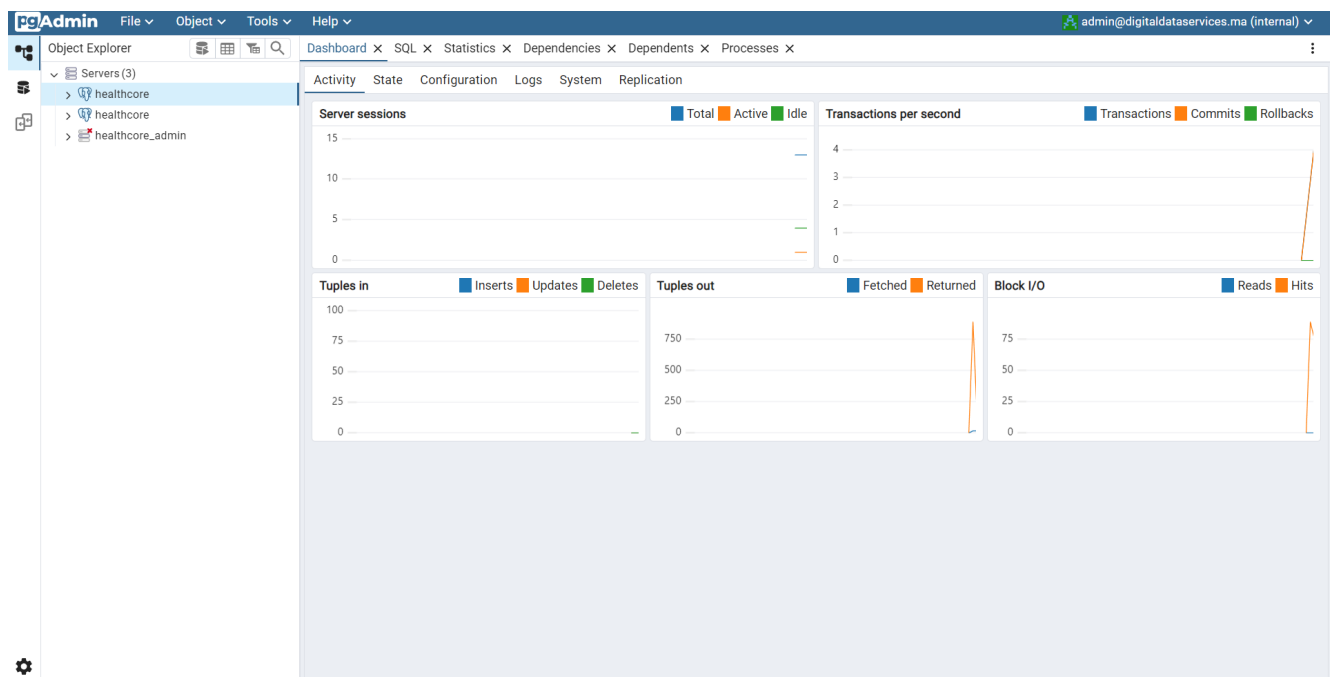


Figure 4.11: pgAdmin attached to the PostgreSQL cluster

4.2.7 Persistent Volumes (PVCs)

Stateful components (Harbor, PostgreSQL, Redis, MinIO) use PersistentVolumeClaims (PVCs) backed by the local-path storage class in this environment. The kubectl view confirms all critical PVCs are Bound with the expected capacity and access mode.

```
anwar_tamasna@vmi2724118:~$ kubectl get pvc -A
```

NAMESPACE	NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
harbor	data-harbor-redis-0	Bound	pvc-679c6741-b162-46b5-8697-54094adcd446	1Gi	RWO	local-path	<unset>	16d
harbor	data-harbor-trivy-0	Bound	pvc-eb543cb8-606f-43dd-89d6-a2c4d418aa98	5Gi	RWO	local-path	<unset>	16d
harbor	database-data-harbor-database-0	Bound	pvc-44a7140f-d92c-4361-8c7c-5301fdad01a8	1Gi	RWO	local-path	<unset>	16d
harbor	harbor-jobservice	Bound	pvc-20a79991-5011-4c84-a22e-59959de0e44c	1Gi	RWO	local-path	<unset>	16d
harbor	harbor-registry	Bound	pvc-a7beba3d-58c6-4080-ad4a-446cfeb90b4c	5Gi	RWO	local-path	<unset>	16d
minio	data-minio-0	Bound	pvc-fb0a0834-9f58-48ff-89fd-8944a2feecd4	10Gi	RWO	local-path	<unset>	39h
minio	data-minio-1	Bound	pvc-91e27b3b-8a6a-49f8-a522-05f5a831dfad	10Gi	RWO	local-path	<unset>	39h
minio	data-minio-2	Bound	pvc-7924a4cd-8f3f-4b69-a28f-2b03a38262f0	10Gi	RWO	local-path	<unset>	39h
postgresql	ha-postgres-1	Bound	pvc-e9c1bcb6-37ab-47d5-bf05-89c83abbff68	5Gi	RWO	local-path	<unset>	6d
postgresql	ha-postgres-2	Bound	pvc-03828cd9-284c-4749-8a2b-edfcaab0a55e	5Gi	RWO	local-path	<unset>	6d
postgresql	ha-postgres-3	Bound	pvc-e2892e3d-bedd-4ab0-8438-d4f3bec91a92	5Gi	RWO	local-path	<unset>	6d
redis	data-redis-0	Bound	pvc-aa13e0dd-116b-46cd-bb00-c25a5178335a	5Gi	RWO	local-path	<unset>	35h
redis	data-redis-1	Bound	pvc-d57089f4-9482-4e98-a05f-49733c846e9c	5Gi	RWO	local-path	<unset>	35h
redis	data-redis-2	Bound	pvc-6c24c323-7d5b-4b7b-bd81-aa7a9460f3aa	5Gi	RWO	local-path	<unset>	35h

Figure 4.12: Cluster-wide PVC status for stateful workloads

4.2.8 Workload Layout (NeuroTalk stack)

NeuroTalk components are organized as separate services: API, Celery workers, Redis for caching/queues, Tika for document processing, and Ollama for local LLM inference. This layout simplifies CI/CD and operational boundaries.

```
anwar_tamasna@vmi2724110:~/Neurotalk$ ll
total 36
drwxrwxr-x  9 anwar_tamasna anwar_tamasna 4096 Aug 12 10:12 ./
drwxr-x--- 13 anwar_tamasna anwar_tamasna 4096 Aug 22 15:54 ../
drwxr-xr-x  2 root          root          4096 Aug 12 10:12 api/
drwxr-xr-x  2 root          root          4096 Aug 12 10:12 celery/
drwxrwxr-x  2 anwar_tamasna anwar_tamasna 4096 Aug 19 00:37 minio/
drwxr-xr-x  2 root          root          4096 Aug 12 10:12 ollama/
drwxrwxr-x  3 anwar_tamasna anwar_tamasna 4096 Aug 18 12:51 postgres/
drwxrwxr-x  2 anwar_tamasna anwar_tamasna 4096 Aug 19 00:38 redis/
drwxrwxr-x  2 anwar_tamasna anwar_tamasna 4096 Aug 19 00:38 tika/
anwar_tamasna@vmi2724110:~/Neurotalk$ |
```

Figure 4.13: NeuroTalk stack directory structure

4.2.9 MinIO Kubernetes Manifests

The MinIO deployment is defined via manifests (Deployment/StatefulSet, Secret, Ingress). Secrets store access and console credentials; the Ingress exposes the console/API over HTTPS (behind the cluster ingress controller).

```
anwar_tamasna@vmi2724110:~/Neurotalk/minio$ ll
total 20
drwxrwxr-x 2 anwar_tamasna anwar_tamasna 4096 Aug 19 00:37 ./
drwxrwxr-x 9 anwar_tamasna anwar_tamasna 4096 Aug 12 10:12 ../
-rw-rw-r-- 1 anwar_tamasna anwar_tamasna  522 Aug 19 00:37 minio-ingress.yaml
-rw-rw-r-- 1 anwar_tamasna anwar_tamasna   165 Aug 19 00:35 minio-secret.yaml
-rw-rw-r-- 1 anwar_tamasna anwar_tamasna  1667 Aug 19 00:36 minio_deploy.yaml
anwar_tamasna@vmi2724110:~/Neurotalk/minio$ |
```

Figure 4.14: MinIO Kubernetes manifests

4.2.10 PostgreSQL and pgAdmin Manifests

PostgreSQL and pgAdmin are deployed via manifests that define storage (PVCs), services, and optional Ingress. For HA and operational automation, CloudNativePG or similar operators can be used to manage upgrades and failover.

```
anwar_tamasna@vmi2724110:~/Neurotalk$ cd ./postgres/  
anwar_tamasna@vmi2724110:~/Neurotalk/postgres$ ls  
cloudnativepg-operator.yaml  ha-postgres.yaml  pgadmin-ingress.yaml  secret-superuser.yaml  
ha-postgres-service.yaml    pgadmin-deployment.yaml  secret-appuser.yaml  
anwar_tamasna@vmi2724110:~/Neurotalk/postgres$ pwd  
/home/anwar_tamasna/Neurotalk/postgres  
anwar_tamasna@vmi2724110:~/Neurotalk/postgres$ |
```

Figure 4.15: PostgreSQL and pgAdmin deployment resources

4.2.11 Security and Operations Highlights

We enforce RBAC at the namespace/project level, use private images from Harbor, and store credentials in Kubernetes Secrets. Persistent volumes are provisioned per stateful service. Monitoring/alerting is integrated via the cluster stack, and backups are stored on MinIO with lifecycle policies.

4.3 Benchmark

This snapshot comes from the Rancher Cluster Dashboard and summarizes current capacity and health. It provides a fast read on performance headroom for the Kubernetes platform.

Environment

K3s **v1.32.6+k3s1** (amd64), cluster age **25 days**, **1** node, **31** deployments, **386** total resources.

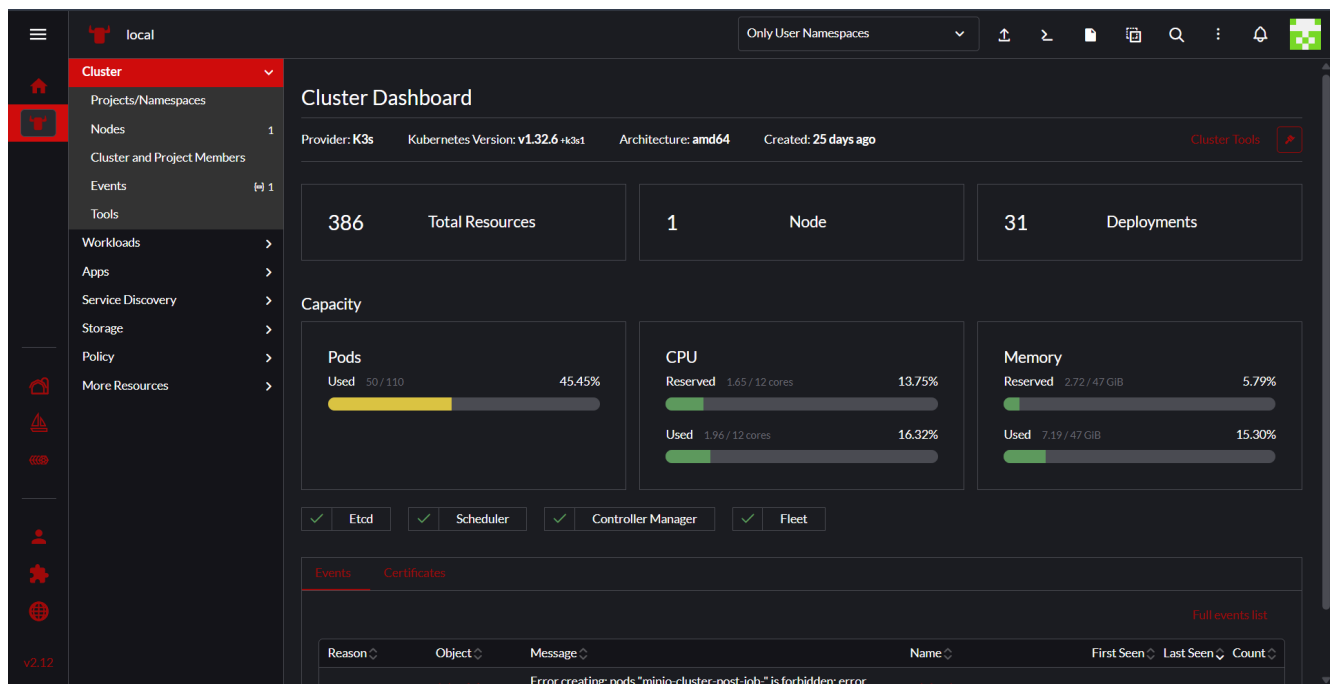


Figure 4.16: Cluster dashboard snapshot (Rancher)

Instant Utilization

- Pods: 50 / 110 (45.45%)
- CPU: reserved 1.65 / 12 (13.75%); used 1.96 / 12 (16.32%)
- Memory: reserved 2.72 / 47 GiB (5.79%); used 7.19 / 47 GiB (15.30%)
- Control plane: Etcd, Scheduler, Controller Manager, Fleet — **healthy**

Performance Interpretation

- Low CPU (~16%) and memory (~15%) usage indicate ample headroom; short spikes and moderate scale-outs will not contend for resources on this node.
- Pod usage at 45% (50/110) leaves capacity for ~60 additional pods, assuming similar resource profiles.
- Reserved vs. used shows minimal overcommit (13.75% reserved), so current quotas/limits are conservative and safe.

- Control-plane components are green, suggesting stable scheduling, persistence, and cluster state.

Quick Requirements Check

- Health budget: core services healthy — **Pass**
- Headroom: CPU, memory, pods all $< 50\%$ — **Pass**
- Overcommit: reserved CPU/memory $< 25\%$ — **Pass**

4.4 Conclusion

In summary, the Rancher Cluster Dashboard provides a comprehensive overview of the current state of the Kubernetes environment. The utilization metrics indicate that the cluster is operating well within its capacity limits, with ample headroom for future growth. The deployment strategies for key components like NeuroTalk, MinIO, PostgreSQL, and pgAdmin are well-defined, leveraging Kubernetes best practices for security, scalability, and maintainability. Overall, the cluster is in a healthy state, ready to support the demands of the applications it hosts.

General Conclusion

I am very pleased to have completed my internship at **Digital Data Service**, where I contributed to the development and deployment of innovative solutions with a strong focus on **Kubernetes (K3s)**, containerization, and modern DevOps practices. This experience allowed me to gain a deeper understanding of the technical and organizational challenges of building scalable, reliable, and secure infrastructures for digital platforms.

From a technical standpoint, the project delivered a cohesive, production-like platform that unifies cluster management, secure image delivery, persistence, and automated deployments. Key outcomes:

- **K3s + Rancher**: centralized cluster ops with namespaces and RBAC.
- **GitOps with ArgoCD**: declarative, auditable, and reproducible releases.
- **NGINX Ingress + Cert-Manager**: secure exposure with automated TLS.
- **Harbor + persistence**: private registry; MinIO, PostgreSQL HA, Redis, Tika where needed.
- **App staging**: NeuroTalk and HealthCore validated with persistent data and env configs.

Lessons learned:

- Favor **declarative** config and version control for all infra artifacts.
- Build **observability** early (metrics, logs, alerts) to shorten feedback loops.

Constraints and mitigations:

- Time and hardware limited scope; used local-path storage pragmatically.
- Drift curbed by **GitOps**; rollback safety via versioned releases/immutable images.

- Stateful reliability via PVCs, health probes, and periodic backups.

Next steps (roadmap):

- Expand monitoring (**Prometheus, Grafana, Loki**) and SLO-based alerting.
- Strengthen security (OPA/Gatekeeper, Cosign, External Secrets + Vault).
- Improve **backup/DR** with Velero and tested runbooks.
- Add autoscaling on custom metrics and safer rollouts (blue/green, canary).
- Explore multi-zone and multi-node control planes; enforce NetworkPolicies.

In conclusion, the platform establishes a solid, scalable foundation. With incremental investments in monitoring, security, and resilience, it is well positioned to operate reliably at production scale.

BIBLIOGRAPHY

1. Digital Data Service — Technical fact sheet (PDF). Internal document.
2. K3s image. <https://docs.k3s.io/>. Accessed: 22-08-25.
3. Rancher brand guidelines (logo). <https://www.rancher.com/brand-guidelines>. Accessed: 22-08-25.
4. Argo CD image. <https://gitlab.com/gitlab-com/gl-infra/k8s-workloads/argocd>. Accessed: 22-08-25.
5. Harbor logo. <https://seeklogo.com/vector-logo/447018/harbor>. Accessed: 22-08-25.
6. NGINX icon. <https://techicons.dev/icons/nginx>. Accessed: 22-08-25.
7. MinIO compliance (logo). <https://www.min.io/compliance>. Accessed: 22-08-25.
8. PostgreSQL logo. https://commons.wikimedia.org/wiki/File:Postgresql_elephant.svg. Accessed: 22-08-25.
9. YAML file icon. https://fr.pngtree.com/freepng/yaml-file-document-icon_4177017.html. Accessed: 22-08-25.
10. Bash logo. <https://bashlogo.com/>. Accessed: 22-08-25.
11. Helm logo. <https://www.pngegg.com/en/png-vensc>. Accessed: 22-08-25.